



Древняя магия каждый день

Пять принципов FP в пяти языках

Павел Аргентов

Evrone.ru

FP Guru

Павел Аргентов

- много опыта
- разрабатывал всё
- FP с 2002 г.

[Evrone.ru](#): разработка
разработчиков

ФП: пять конкретных свойств кода

Смысл: описание результата, а не микроменеджмент процесса.

- `[1,2,3].map(x ⇒ x * 2)`
- `const add = x ⇒ y ⇒ x + y`
- `const x = 1; x = 2 // TypeError`

Пять языков

OCaml

язык богов (тм)

Scala

почти OCaml

Python

язык МЧС

JS

вездесущ

Rust

всемогущ

Декларативность

Декларативность

- Императивный код — микроменеджмент.
- Декларативный код — спецификация.

SELECT ... WHERE vs for ... in
for ... end vs list.map

Декларативность

- Людям — понятность.
- Машинам — законы оптимизации.

* для работы понадобится все остальные свойства FP

Декларативность: задача

Сумма скидок для заказов дороже 100, скидка 10%

Декларативность: OCaml

```
let total_discount orders =  
  orders  
  ▷ filter (fun o → o.price > 100.0)  
  ▷ map (fun o → o.price *. 0.1)  
  ▷ fold_left ( +. ) 0.0
```

Декларативность: Scala

```
def totalDiscount(orders: List[Order]): Double =  
  orders  
    .filter(_ .price > 100.0)  
    .map(_ .price * 0.1)  
    .sum
```

Декларативность: Rust

```
fn total_discount(orders: &[Order]) → f64 {  
    orders.iter()  
        .filter(|o| o.price > 100.0)  
        .map(|o| o.price * 0.1)  
        .sum()  
}
```

Декларативность: Python

```
def total_discount(orders):  
    return sum(o.price * 0.1 for o in orders if o.price > 100.0)
```

Декларативность: JS

```
const totalDiscount = (orders) =>  
  orders  
    .filter((o) => o.price > 100)  
    .map((o) => o.price * 0.1)  
    .reduce((acc, d) => acc + d, 0);
```

Выражения вместо инструкций

Выражения вместо инструкций

- Инструкции производят эффект
- Выражения вычисляются в значения
- Значения можно компоновать

Выражения вместо инструкций: задача

Классификация числа — вернуть метку без промежуточной переменной.

Выражения вместо инструкций: OCaml

```
let classify n =  
  match n with  
  | n when n < 0 → "negative"  
  | 0             → "zero"  
  | _             → "positive"
```

```
let label = classify (-5)  
  ▷ String.uppercase_ascii
```

Выражения вместо инструкций: Scala

```
def classify(n: Int): String =  
  n match {  
    case n if n < 0 => "negative"  
    case 0          => "zero"  
    case _          => "positive"  
  }  
  
val label = classify(-5).toUpperCase
```

Выражения вместо инструкций: Rust

```
fn classify(n: i32) → &'static str {  
    match n {  
        n if n < 0 ⇒ "negative",  
        0          ⇒ "zero",  
        _          ⇒ "positive",  
    }  
}  
  
let label = classify(-5).to_uppercase();
```

Выражения вместо инструкций: Python

```
def classify(n: int) → str:  
    return (  
        "negative" if n < 0 else  
        "zero"     if n == 0 else  
        "positive"  
    )  
label = classify(-5).upper()  
# match n: ... → match – инструкция
```

Выражения вместо инструкций: JS

```
const classify = (n) =>  
  n < 0 ? "negative"  
  : n === 0 ? "zero"  
  : "positive";
```

```
const label = classify(-5).toUpperCase();  
// if (...) {} → инструкция
```



Павел Аргентов

Tg: @argent_smith